

CHAPITRE 3

Créer un objet

Ce que vous allez apprendre dans ce chapitre :

- Comprendre le concept d'objet
- Maîtriser les concepts d'instanciation d'une classe et la destruction d'un objet



01 heure



CHAPITRE 3

Créer un objet

1. Définition d'un objet
2. Instanciation
3. Destruction explicite d'un objet



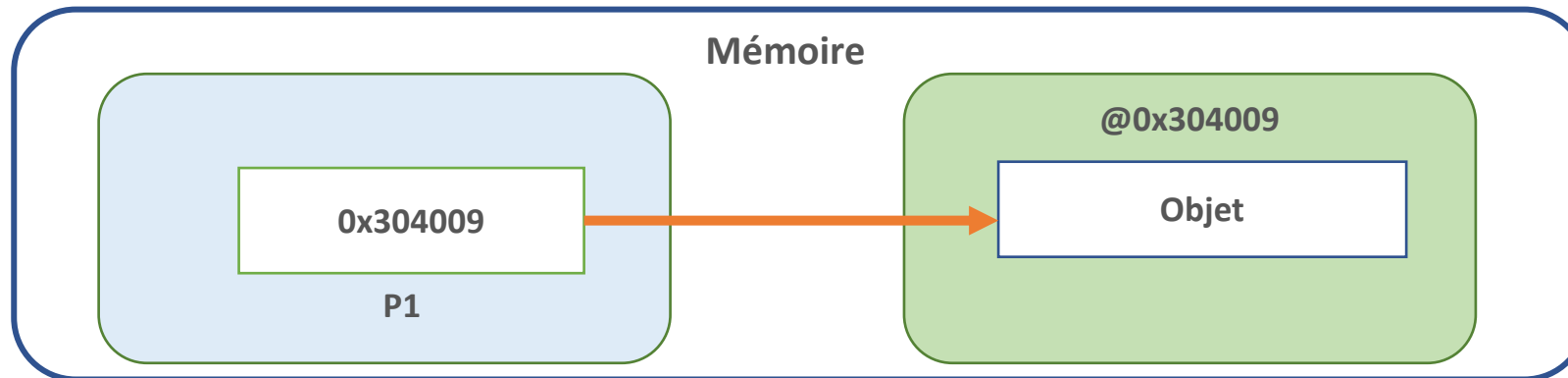
03 - Créer un objet

Définition d'un objet

Notion d'objet

OBJET= Référent + Etat + Comportement

- Chaque objet doit avoir un nom (référence) « qui lui est propre » pour l'identifier
- La référence permet d'accéder à l'objet, mais n'est pas l'objet lui-même. Elle contient l'adresse de l'emplacement mémoire dans lequel est stocké l'objet.

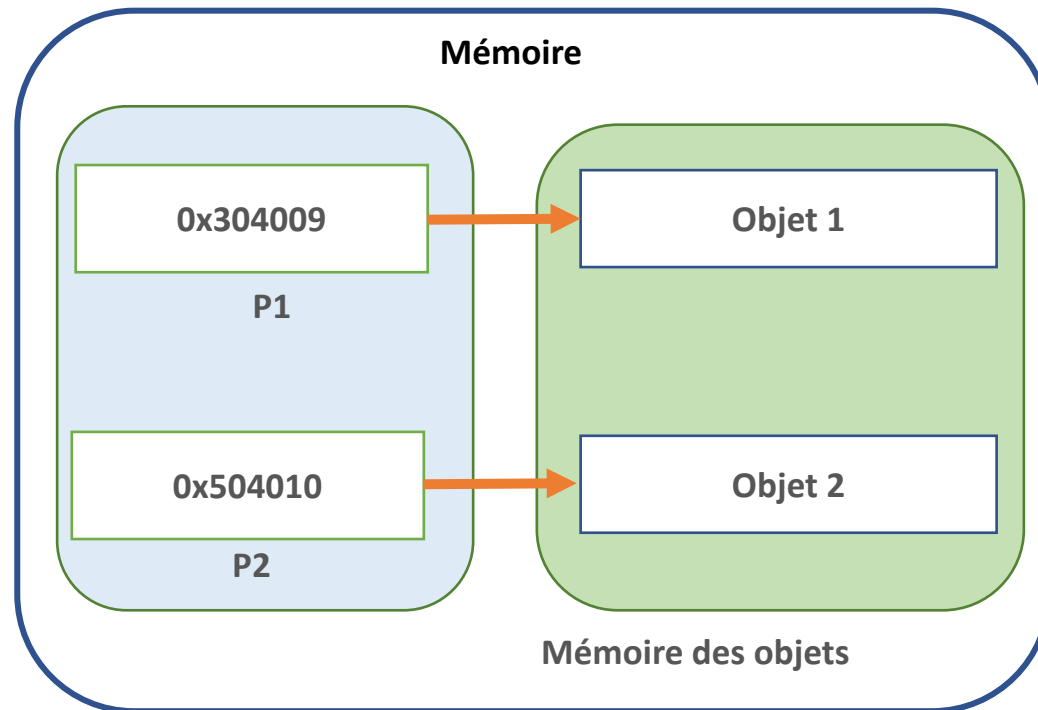


03 - Créer un objet

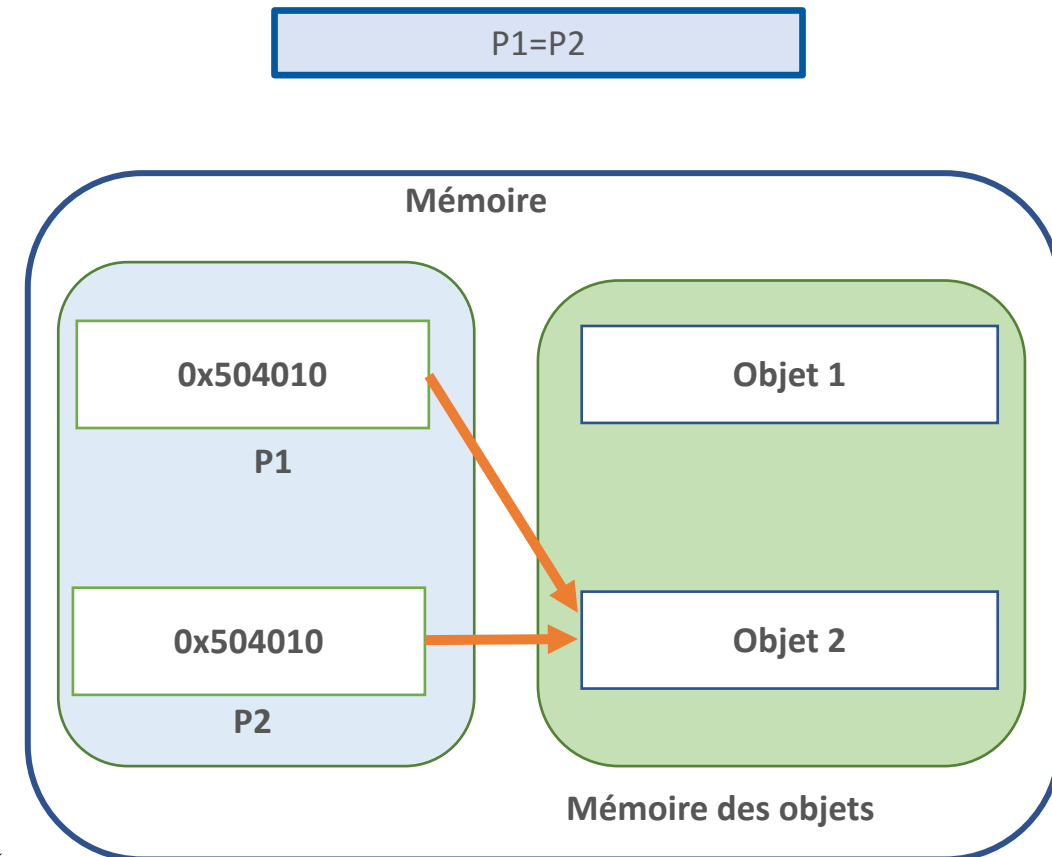
Définition d'un objet

Notion d'objet

- Plusieurs référents peuvent référer un même objet → **Adressage indirect**



Recopie l'adresse de P2 dans le référent de P1



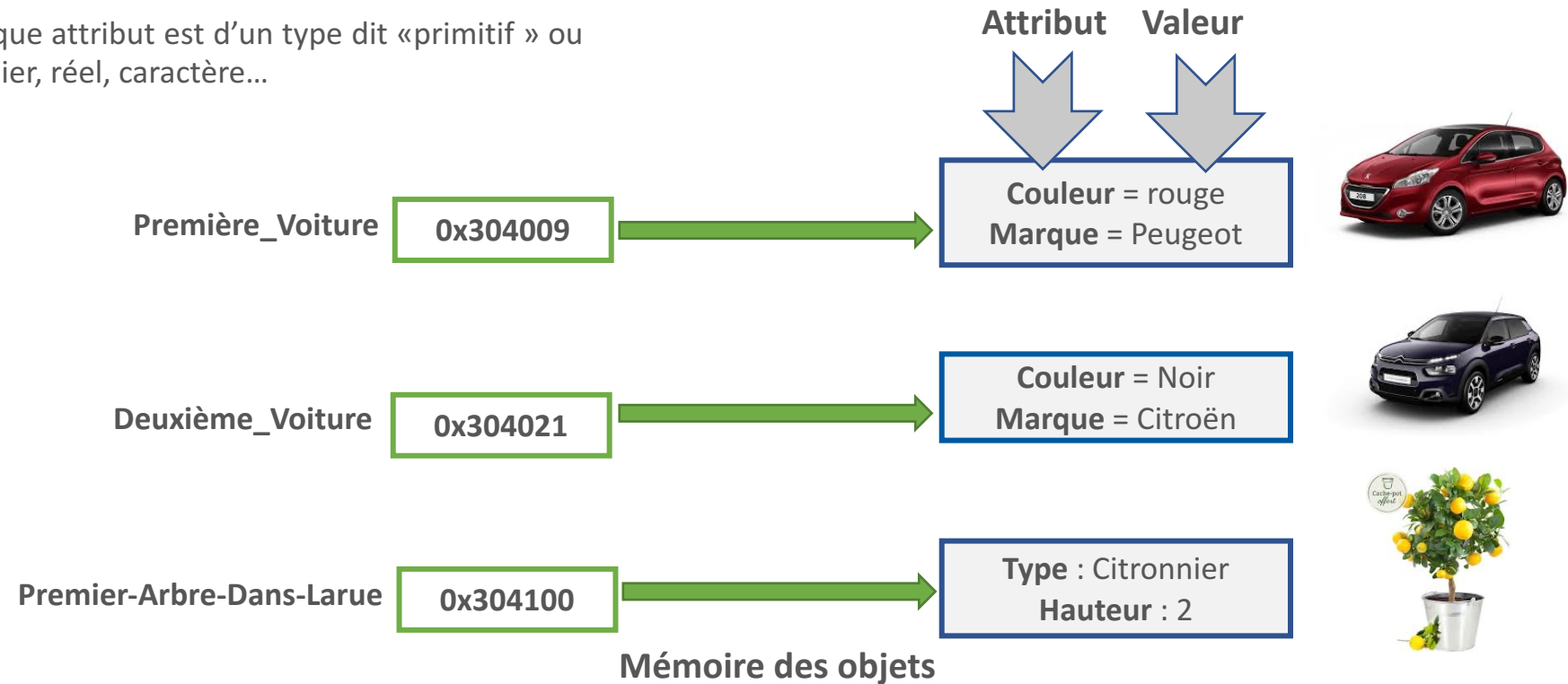
03 - Créer un objet

Définition d'un objet

Notion d'objet

- Un objet est capable de sauvegarder un état c'est-à-dire un ensemble d'information dans les **attributs**
- Les **attributs** sont l'ensemble des informations permettant de représenter l'état de l'objet
- Exemple d'objets où chaque attribut est d'un type dit «primitif » ou « prédéfini », comme entier, réel, caractère...

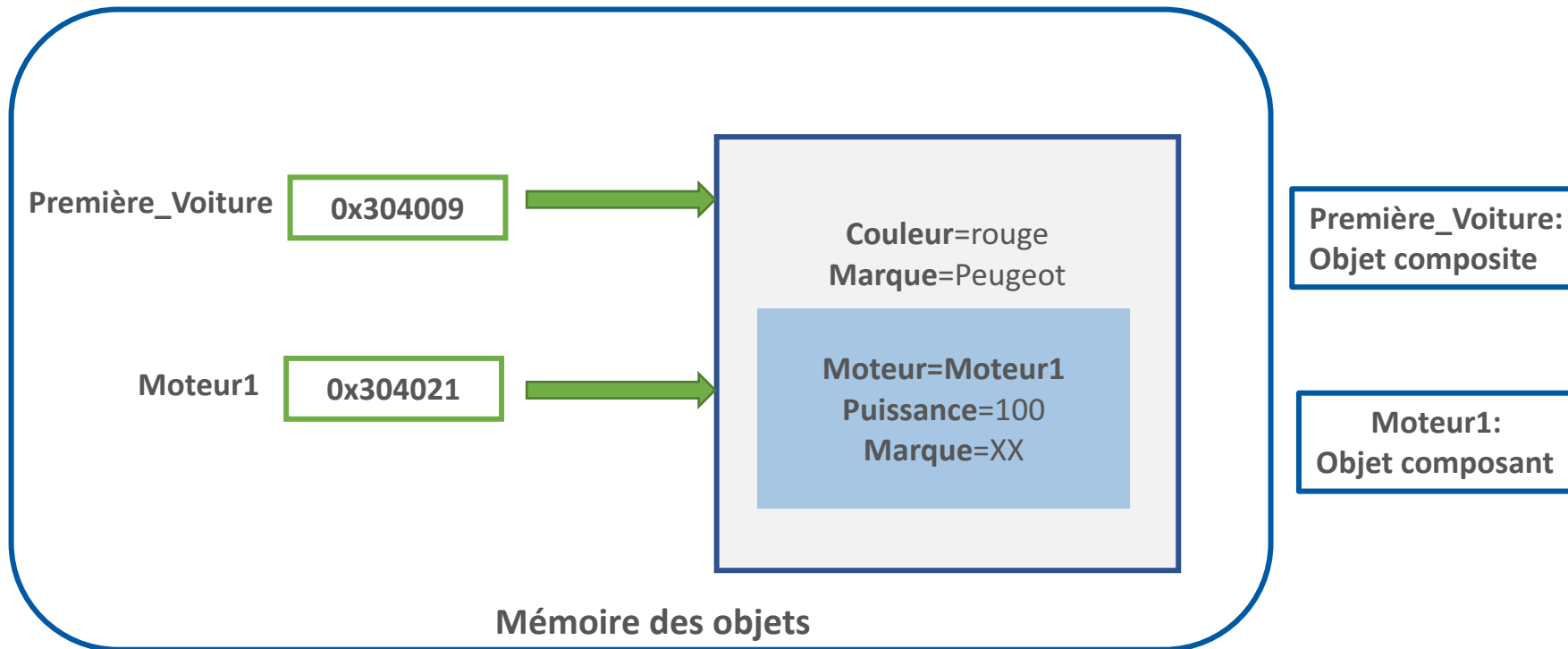
OBJET= Référent + Etat + Comportement



03 - Créer un objet

Définition d'un objet

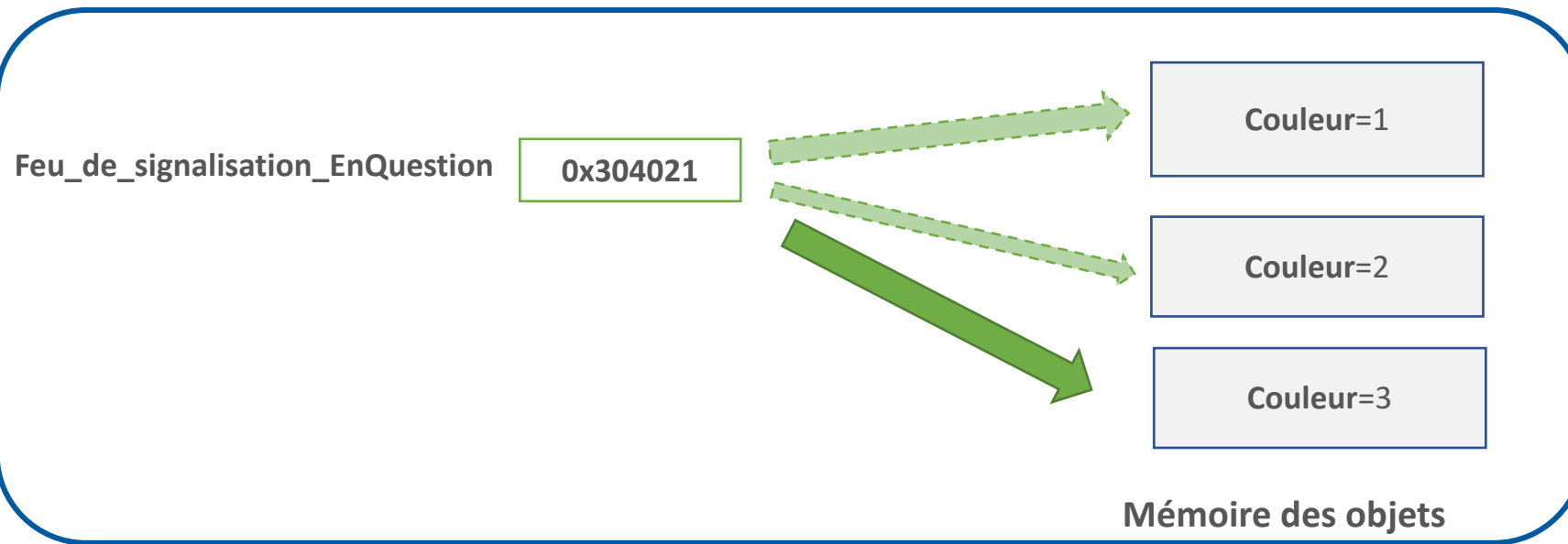
- Un objet stocké en mémoire peut être placé à l'intérieur de l'espace mémoire réservé à un autre. Il s'agit d'un **Objet Composite**



03 - Créer un objet

Définition d'un objet

- Les objets changent d'état
- Le cycle de vie d'un objet se limite à une succession de changements d'états
- Soit l'objet Feu_de_signalisation_EnQuestion avec sa couleur prenant trois valeurs

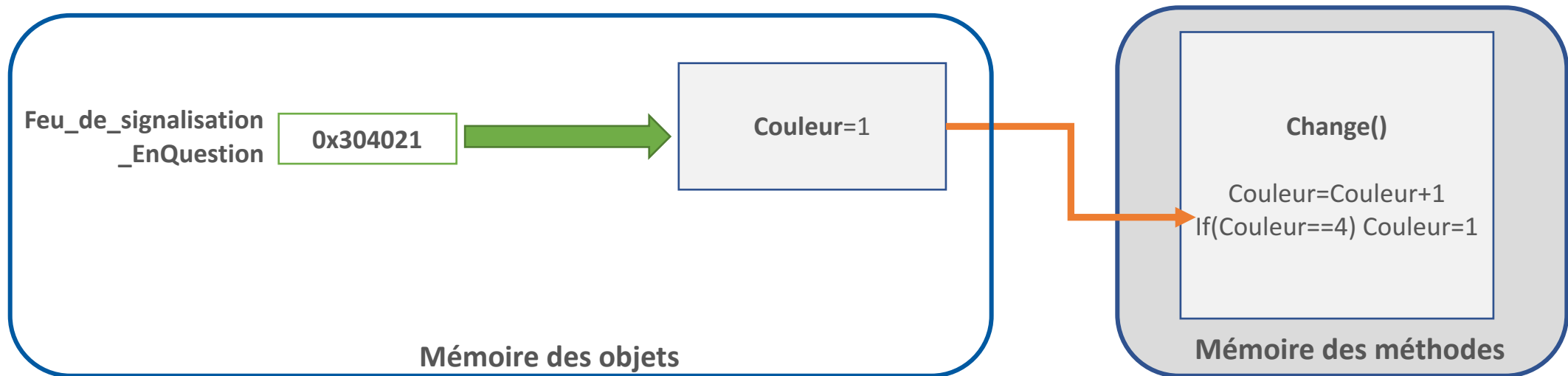


03 - Créer un objet

Définition d'un objet

OBJET= Référent + Etat + Comportement

Mais quelle est la cause des changements d'états ?
– LES METHODES



CHAPITRE 3

Créer un objet

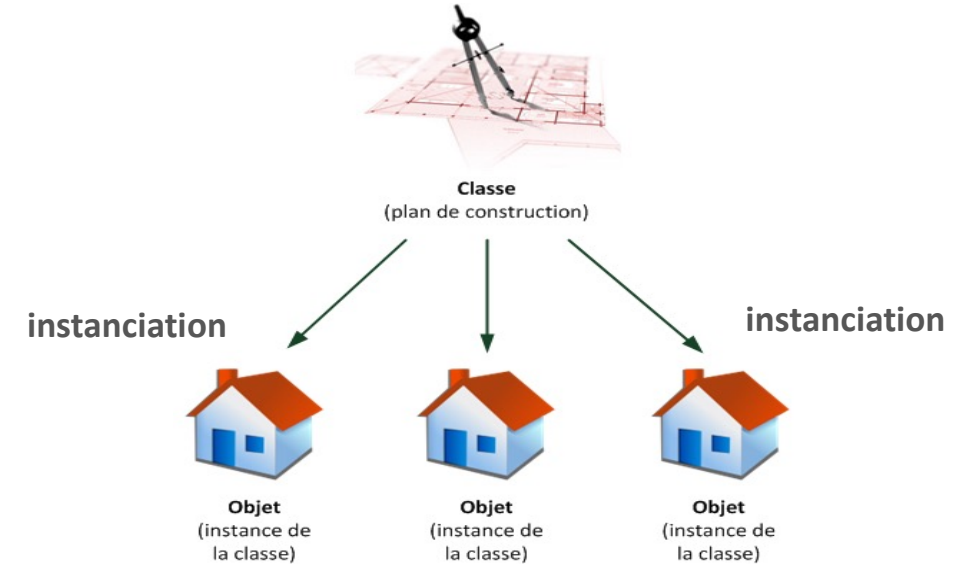
1. Définition d'un objet
2. **Instanciation**
3. Destruction explicite d'un objet



03 - Créer un objet

Instanciation

- L'instanciation d'un objet est constituée de deux phases :
 - **Une phase de ressort de la classe** : créer l'objet en mémoire
 - **Une phase du ressort de l'objet** : initialiser les attributs
- L'instanciation explicite d'un objet se fait via un **constructeur**. Il est appelé automatiquement lors de la création de l'objet dans la mémoire.



CHAPITRE 3

Créer un objet

1. Définition d'un objet
2. Instanciation
3. **Destruction explicite d'un objet**



03 - Créer un objet

Destruction explicite d'un objet



- **Rappel** : un destructeur est une méthode particulière qui permet **la destruction d'un objet non référencé**
 - La destruction explicite d'un objet s'effectue en faisant appel au destructeur de la classe

CHAPITRE 4

Connaitre l'encapsulation

Ce que vous allez apprendre dans ce chapitre :

- Comprendre le principe d'encapsulation
- Connaitre les modificateurs et les accesseurs d'une classe : Syntaxe et utilité



01 heure



CHAPITRE 4

Connaitre l'encapsulation

1. Principe de l'encapsulation
2. Modificateurs et accesseurs (getters, setters, ...)

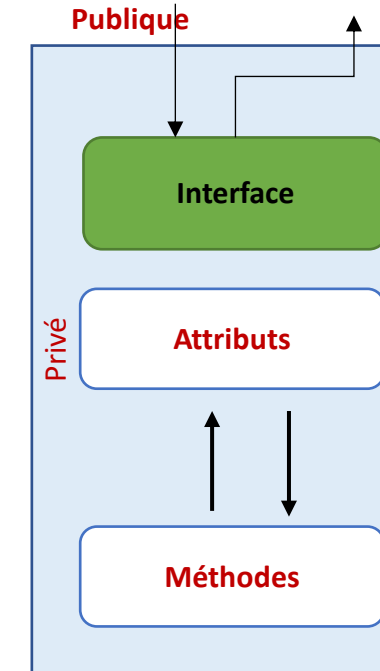
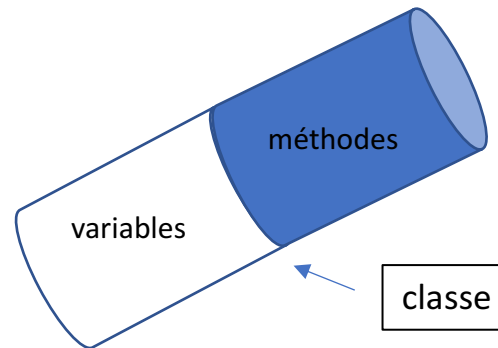


04 - Connaitre l'encapsulation

Principe de l'encapsulation

Principe de l'encapsulation

- L'encapsulation est le fait de réunir à l'intérieur d'une même entité (objet) le code (méthodes) + données (attributs)
- L'encapsulation consiste à protéger l'information contenue dans un objet
- **Il est donc possible de masquer les informations d'un objet aux autres objets.**
- L'encapsulation consiste donc à masquer les détails d'implémentation d'un objet, en définissant une **interface**
- L'interface est **la vue externe d'un objet**, elle définit les services **accessibles** (Attributs et Méthodes) aux utilisateurs de l'objet
 - **interface = liste des signatures des méthodes accessibles**
 - **Interface = Carte de visite de l'objet**



04 - Connaitre l'encapsulation

Principe de l'encapsulation

- Les objets ne restreignent leur accès qu'aux méthodes de leur classe

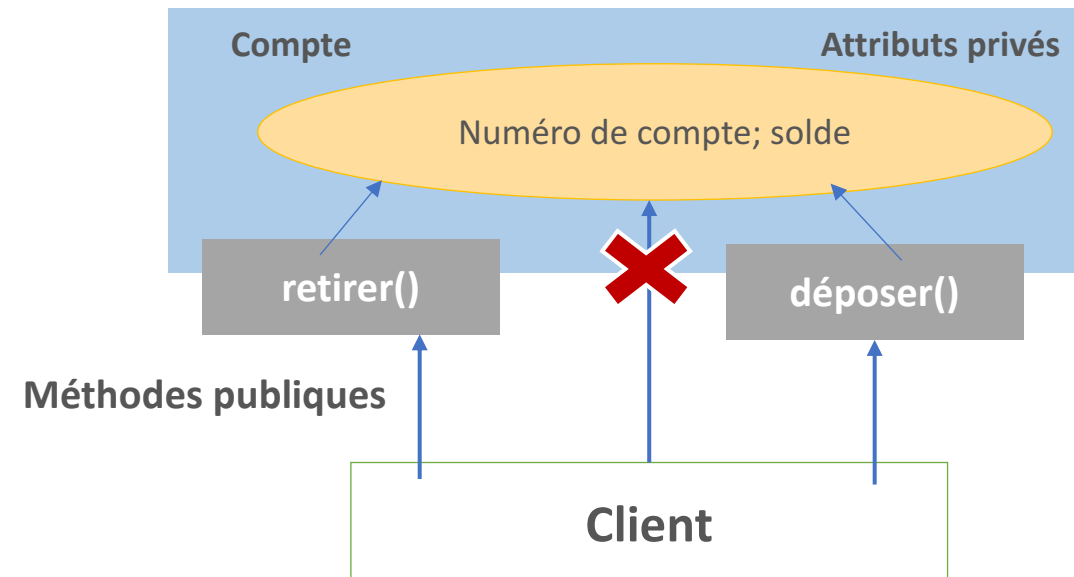
→ Ils protègent leurs attributs

- Cela permet d'avoir un contrôle sur tous les accès

Exemple :

- Les attributs de la classe compte sont privés
- Ainsi le solde d'un compte n'est pas accessible par un client qu'à travers les méthodes retirer() et déposer() qui sont publiques

→ Un client ne peut modifier son solde qu'en effectuant une opération de dépôt ou de retrait



CHAPITRE 4

Connaitre l'encapsulation

1. Principe de l'encapsulation
2. **Modificateurs et accesseurs (getters, setters, ...)**



04 - Connaitre l'encapsulation

Modificateurs et accesseurs (getters, setters, ...)

Modificateurs et accesseurs

- Pour interagir avec les attributs d'un objet de l'extérieur, il suffit de créer des méthodes publiques dans la classe appelée **Accesseurs et Modificateurs**
- **Accesseurs (getters):** Méthodes qui retournent la valeur d'un attribut d'un objet (un attribut est généralement privé)
- La notation utilisée est **getXXX** avec XXX l'attribut retourné
- **Modificateurs (setters):** Méthodes qui modifient la valeur d'un attribut d'un objet
- La notation utilisée est **setXXX** avec XXX l'attribut modifié

Voiture
Couleur: chaîne Marque: chaîne

Première_Voiture
Couleur=rouge Marque=Peugeot

getCouleur()

Quelle est la couleur de Première_Voiture?

Rouge

Première_Voiture
Couleur=rouge Marque=Peugeot

setCouleur(Bleu)

Couleur = bleu

Couleur = Bleu

CHAPITRE 5

Manipuler les méthodes

Ce que vous allez apprendre dans ce chapitre :

- Manipuler les méthodes : définition et appel
- Différencier entre méthode d'instance et méthode de classe



01 heure

CHAPITRE 5

Manipuler les méthodes

1. Définition d'une méthode
2. Visibilité d'une méthode
3. Paramètres d'une méthode
4. Appel d'une méthode
5. Méthodes de classe

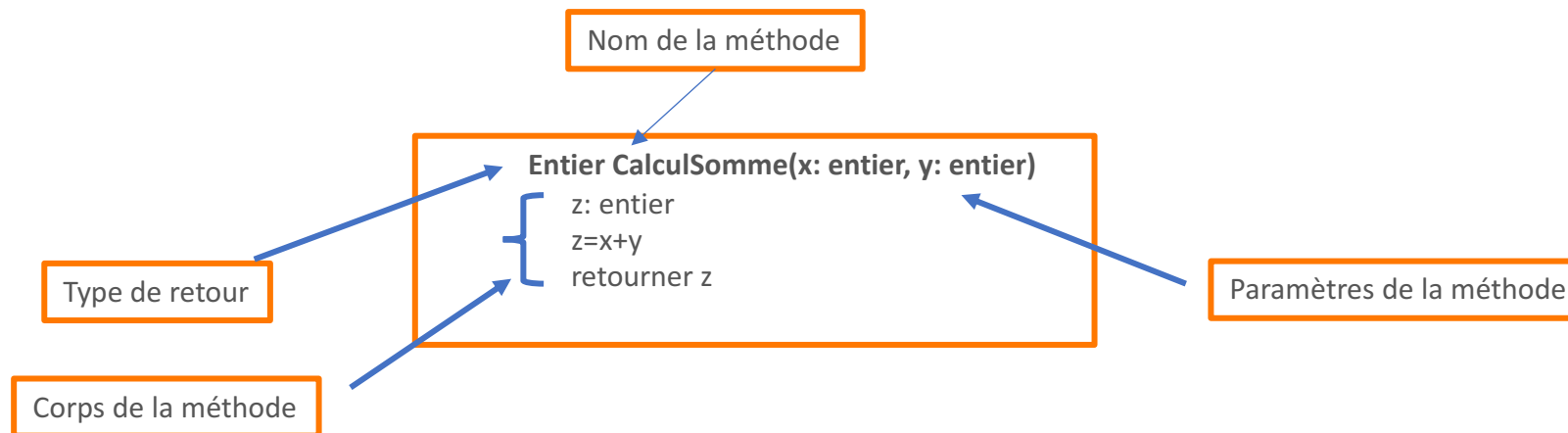


05 - Manipuler les méthodes

Définition d'une méthode

- La définition d'une méthode ressemble à celle d'une procédure ou d'une fonction. Elle se compose d'une entête et un bloc.
- **L'entête** précise :
 - **Un nom**
 - **Un type de retour**
 - Une liste (éventuellement vide) de **paramètres** typés en entrée
 - Le **bloc** est constitué d'une suite d'instructions (un bloc d'instructions) qui constituent le corps de la méthode

Exemple de syntaxe :



CHAPITRE 5

Manipuler les méthodes

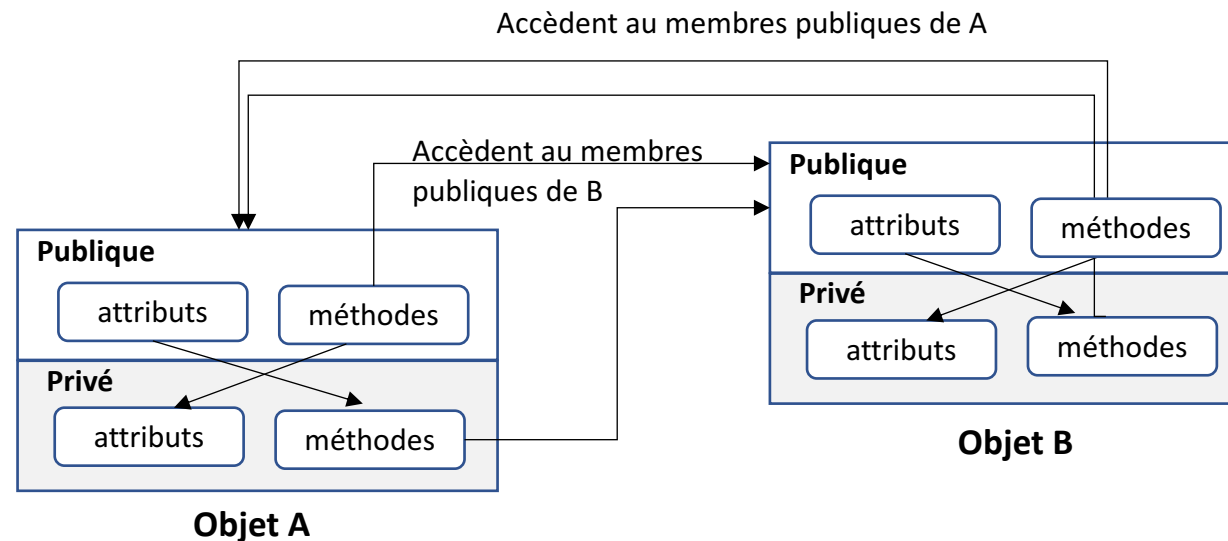
1. Définition d'une méthode
2. **Visibilité d'une méthode**
3. Paramètres d'une méthode
4. Appel d'une méthode
5. Méthodes de classe



05 - Manipuler les méthodes

Visibilité d'une méthode

- La visibilité d'une méthode définit les droits d'accès autorisant l'appel aux méthodes de la classe
- Publique (+) :**
 - L'appel de la méthode est autorisé aux méthodes de toutes les classes
- Protégée (#) :**
 - L'appel de la méthode est réservé aux méthodes des classes héritières
(A voir ultérieurement dans la partie Héritage)
- Privée(-) :**
 - L'appel de la méthode est limité aux méthodes de la classe elle-même



CHAPITRE 5

Manipuler les méthodes

1. Définition d'une méthode
2. Visibilité d'une méthode
- 3. Paramètres d'une méthode**
4. Appel d'une méthode
5. Méthodes de classe



05 - Manipuler les méthodes

Paramètres d'une méthode



- Il est possible de passer des arguments (appelés aussi **paramètres**) à une **méthode**, c'est-à-dire lui fournir le nom d'une variable afin que la **méthode** puisse effectuer des opérations sur ces arguments ou bien grâce à ces arguments.

Entier CalculSomme(x: entier, y: entier)

z: entier

z=x+y

retourner z

Paramètres de la méthode

CHAPITRE 5

Manipuler les méthodes

1. Définition d'une méthode
2. Visibilité d'une méthode
3. Paramètres d'une méthode
4. **Appel d'une méthode**
5. Méthodes de classe



05 - Manipuler les méthodes

Appel d'une méthode



- Les méthodes ne peuvent être définies comme des composantes d'une classe.
- Une méthode est appelée pour un objet. Cette méthode est appelée **méthode d'instance**.
- L'appel d'une méthode pour un objet se réalise en nommant l'objet suivi d'un point suivi du nom de la méthode et sa liste d'arguments.

nomObjet.nomMethode(arg1, arg2,..)

Où :

- **nomObjet** : nom de la référence à l'objet
- **nomMéthode** : nom de la méthode

Exemple : la classe *Véhicule*

<i>Véhicule</i>
<i>Marque :string</i>
<i>Puissance :integer</i>
<i>Vitesse-max :integer</i>
<i>Vitesse-courante :ini</i>
<i>Démarrer()</i>
<i>Accélérer()</i>
<i>Avancer()</i>
<i>Reculer()</i>

v est objet de type Véhicule
v.Démarrer()

CHAPITRE 5

Manipuler les méthodes

1. Définition d'une méthode
2. Visibilité d'une méthode
3. Paramètres d'une méthode
4. Appel d'une méthode
5. **Méthodes de classe**



05 - Manipuler les méthodes

Méthodes de classe

- Méthode de classe est une méthode déclarée dont l'exécution ne dépend pas des objets de la classe
- Étant donné qu'elle est liée à la classe et non à ses instances, on préfixe le nom de la méthode par le nom de la classe

nomClasse.nomMethode(arg1, arg2,..)

Exemple :



X
Entier CalculSomme(x: entier, y: entier)

X.CalculSomme (2,5)

- Puisque les méthodes de classe appartiennent à la classe, elles ne peuvent en aucun cas accéder aux attributs d'instances qui appartiennent aux instances de la classe
- Les méthodes d'instances accèdent aux attributs d'instance et méthodes d'instance
- Les méthodes d'instances accèdent aux attributs de classe et méthodes de classe
- Les méthodes de classe accèdent aux attributs de classe et méthodes de classe
- Les méthodes de classe n'accèdent pas aux attributs d'instance et méthodes d'instance



PARTIE 2

Connaître les principaux piliers de la POO

Dans ce module, vous allez :

- Maitriser le principe d'héritage en POO
- Maitriser le concept du polymorphisme en POO
- Maitriser le principe de l'abstraction en POO et son utilité
- Manipuler les interfaces (définition et implémentation)



15 heures

CHAPITRE 1

Définir l'héritage

Ce que vous allez apprendre dans ce chapitre :

- Comprendre le principe de l'héritage et distinguer ses types
- Comprendre le principe de surcharge de constructeur
- Maitriser le chainage des constructeurs
- Maitriser la visibilité des membres d'une classe dérivée



06 heures



CHAPITRE 1

Définir l'héritage

1. Principe de l'héritage
2. Types d'héritage
3. Surcharge des constructeurs et chaînage
4. Visibilité des attributs et des méthodes de la classe fille



01 - Définir l'héritage

Principe de l'héritage



Principe de l'héritage

- L'héritage est une notion qui définit une relation de **spécialisation** ou de **généralisation** entre différentes classes (Exp : une relation d'héritage entre les classes Employé et Directeur, et la classe Salarié).
- Lorsqu'une classe A hérite d'une classe B :
 - A s'octroie toutes les propriétés de B, en plus de ses propres propriétés.
 - A est alors considéré comme une spécialisation de B.
 - Dans le même temps, B peut être vu comme une généralisation de A.
 - A est appelée classe fille de B, ou classe dérivée de B.
 - B est dite classe mère ou super-classe de A.
- **Toute instance (objet) de A peut être considéré comme un objet de B.**
- **Une instance de B ne peut être toujours considéré comme un objet de A.**

01 - Définir l'héritage

Principe de l'héritage



Principe de l'héritage

Exemple :

- Considérons la définition des 3 classes Personne, Etudiant et Employé suivantes :

Personne
Nom: chaîne CIN: entier anneeNaiss: entier
age()

Etudiant
Nom: chaîne CIN: entier anneeNaiss: entier note1, note2: réel
age() moyenne()

Employé
Nom: chaîne CIN: entier anneeNaiss: entier prixHeure: réel nbreHeure: réel
age() Salaire()

Problème :

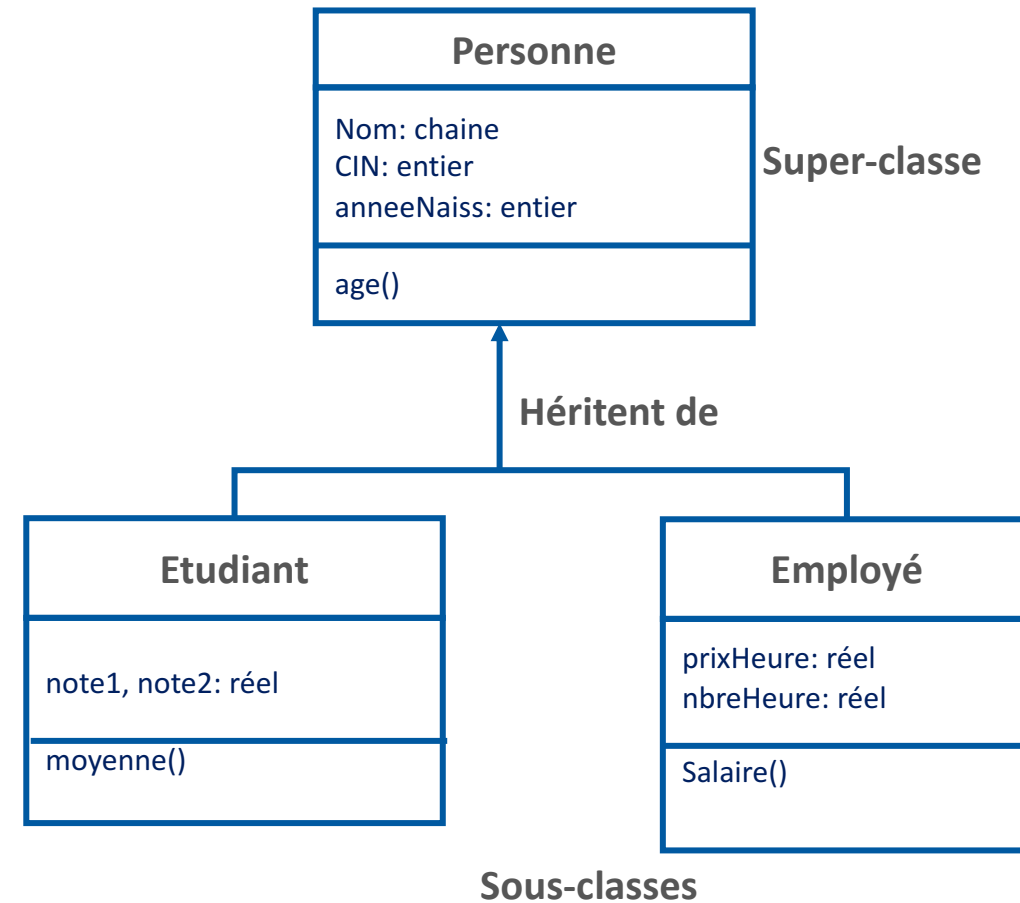
- Duplication du code.
- Toute modification d'un attribut ou d'une méthode doit être répétée dans chaque classe où ils ont été déclarés. (Exp : si on souhaite modifier le type de l'attribut **anneeNaiss**, il faut le faire dans chacune des 3 classes).

01 - Définir l'héritage

Principe de l'héritage

Solution :

- Placer dans la **classe mère** les propriétés en commun à toutes les autres classes.
- On ne garde dans les classes filles que les attributs ou méthodes qui leur sont **spécifiques**.
- Les classes dérivées **héritent** automatiquement des attributs (et des méthodes) qui n'ont pas besoin d'être réécrits.



01 - Définir l'héritage

Principe de l'héritage



Intérêt de l'héritage

- L'héritage permet de factoriser le code en **regroupant les caractéristiques en commun** à plusieurs classes au sein d'une seule (la classe mère).
- **La création de nouvelles classes** s'en trouve facilitée grâce à une hiérarchie bien définie des classes.

CHAPITRE 1

Définir l'héritage

1. Principe de l'héritage
2. **Types d'héritage**
3. Surcharge des constructeurs et chaînage
4. Visibilité des attributs et des méthodes de la classe fille



01 - Définir l'héritage

Types d'héritage

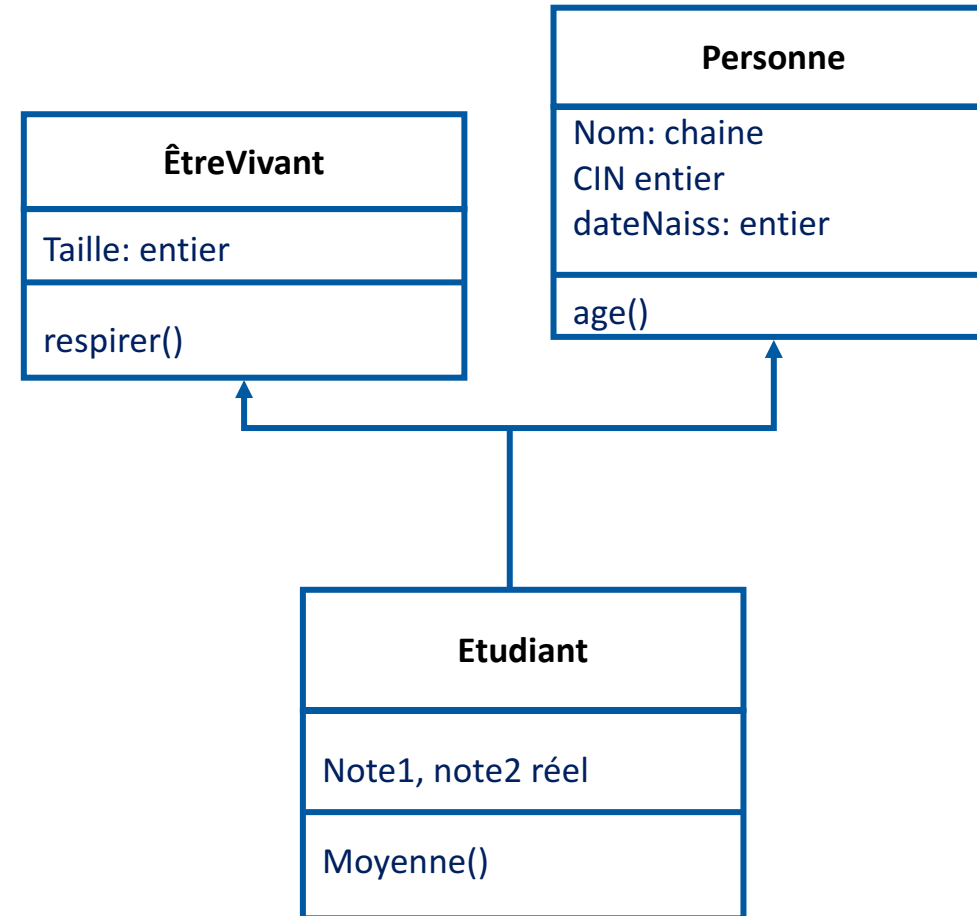
Héritage multiple

- Une classe peut hériter de plusieurs classes

Exemple :

- Un Etudiant est à la fois une Personne et un EtreVivant
- La classe Etudiant hérite des attributs et des méthodes des
- deux classes

→ Il deviennent ses propres attributs et méthodes



01 - Définir l'héritage

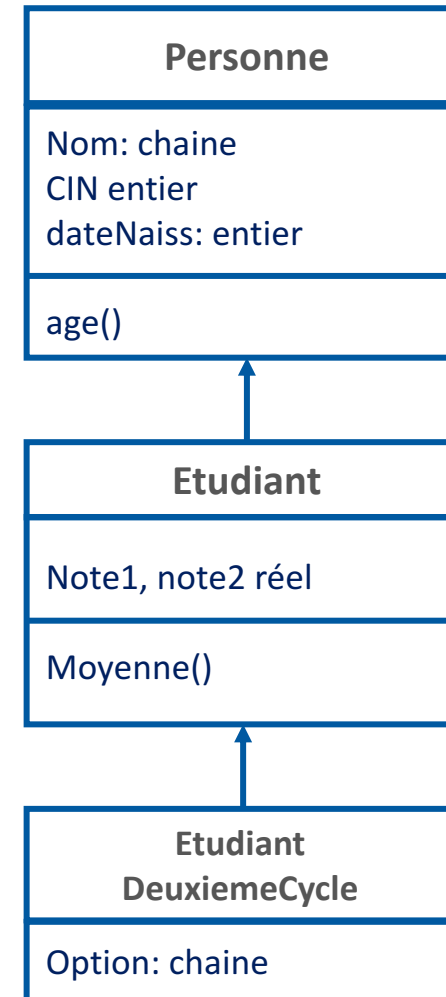
Types d'héritage

Héritage en cascade

- Une classe sous-classe peut être elle-même une super-classe.

Exemple :

- Etudiant hérite de Personne
- EtudiantDeuxièmeCycle hérite de Etudiant
- EtudiantDeuxièmeCycle hérite de Personne



CHAPITRE 1

Définir l'héritage

1. Principe de l'héritage
2. Types d'héritage
- 3. Surcharge des constructeurs et chaînage**
4. Visibilité des attributs et des méthodes de la classe fille



01 - Définir l'héritage

Surcharge des constructeurs et chaînage



Surcharge du constructeur

- Les constructeurs portant le **même nom** mais **une signature différente** sont appelés constructeurs surchargés
- La signature d'un constructeur comprend les types des paramètres et le nombre des paramètres
- Dans une classe, les constructeurs peuvent avoir différents nombres d'arguments, différentes séquences d'arguments ou différents types d'arguments
- Nous pouvons avoir plusieurs constructeurs mais lors de la création d'un objet, le compilateur choisit la méthode qui doit être appelée en fonction du nombre et du type des arguments

Surcharge du
constructeur
CréerPersonne

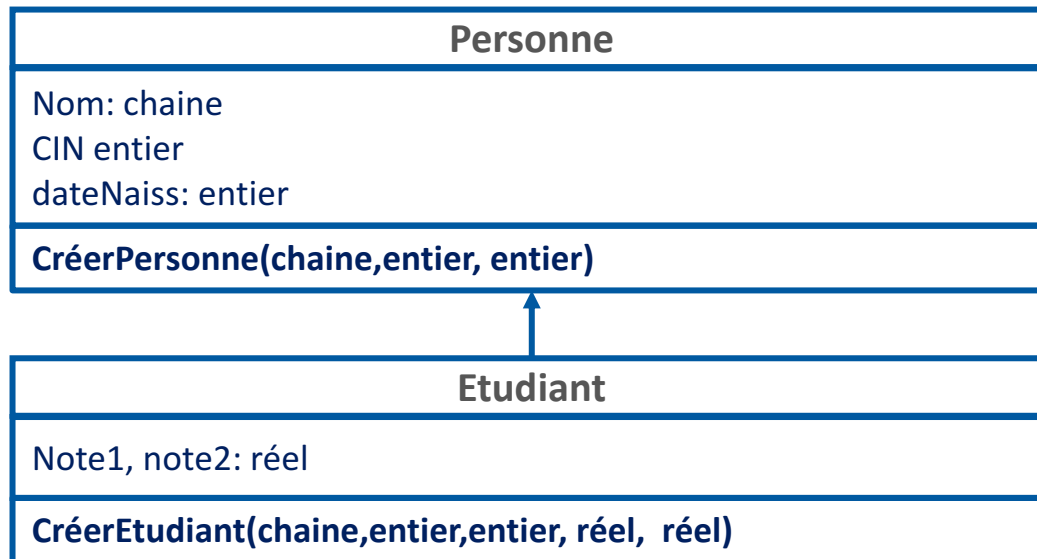
Personne
Nom: chaine CIN: entier dateNaiss: entier
age() CréerPersonne(chaine, entier, entier) CréerPersonne(chaine, entier) CréerPersonne(entier, chaine, entier)

01 - Définir l'héritage

Surcharge des constructeurs et chaînage

Chainage des constructeurs

- Le chaînage de constructeurs est une technique **d'appel d'un constructeur de la classe mère à partir d'un constructeur de la classe fille**
- Le chaînage des constructeurs permet **d'éviter la duplication du code** d'initialisation des attributs qui sont hérités par la classe fille



CréerEtudiant(Nom: chaine, CIN entier, dateNaiss: entier, note1: réel, note2: réel)

CréerPersonne(Nom, CIN, dateNaiss)
note1=15
note2=13.5

appel du constructeur de Personne

Corps du constructeur de Etudiant

CHAPITRE 1

Définir l'héritage

1. Principe de l'héritage
2. Types d'héritage
3. Surcharge des constructeurs et chaînage
4. **Visibilité des attributs et des méthodes de la classe fille**



01 - Définir l'héritage

Visibilité des attributs et des méthodes de la classe fille

Visibilité des membres et héritage

- Si une classe mère définit certains (ou tous) de ses **attributs et méthodes**, comme **privés**, alors aucune de ses **classes filles** ne peut y accéder, ni les modifier.
- Pour permettre à une classe fille de lire ou modifier un attribut hérité, la classe mère doit les déclarer comme **protégés**.

Exemple :

meth1Etudiant()

Nom="X" → incorrect car Nom est un attribut privé dans Personne
CIN=12365 → Correct car CIN est déclaré protégé dans Personne
meth1Personne() → incorrect car la méthode est privée
meth2Personne() → correct car la méthode est protégée

Corps de la méthode meth1Etudiant()

